

Relatório Técnico Detalhado das Alterações de Código

Projeto: MINE-REDE-SOCIAL- **Data:** 20 de Junho de 2026 **Autor:** Manus AI

1. Introdução

Este relatório detalha as intervenções realizadas no código-fonte do projeto MINE-REDE-SOCIAL- com o objetivo de corrigir vulnerabilidades de segurança, melhorar a estabilidade e otimizar a performance. As correções foram implementadas em várias fases, abordando desde regressões críticas introduzidas por alterações anteriores até problemas de segurança e lógica identificados numa auditoria exhaustiva.

2. Resumo das Correções Implementadas

As principais áreas de intervenção e as correções aplicadas incluem:

- **Resolução de Regressão Crítica:** Corrigido um conflito na função `safeJSONParse` que impedia a leitura correta de dados do `localStorage`, causando perda de dados em runtime.
- **Melhoria da Segurança de Autenticação:** Atualizado o sistema de hashing de passwords para uma versão mais robusta, garantindo compatibilidade com utilizadores existentes.
- **Reforço da Segurança DOM:** Eliminado o uso inseguro de `innerHTML` com interpolação direta em componentes de UI, substituindo por métodos de criação de elementos DOM mais seguros.
- **Prevenção de IDOR (Insecure Direct Object Reference):** Corrigida uma vulnerabilidade que permitia a um utilizador manipular notificações de outros utilizadores.
- **Endurecimento de CORS e Cookies:** Reforçada a configuração de Cross-Origin Resource Sharing (CORS) para o Socket.IO e as políticas de `SameSite` para cookies de sessão, mitigando riscos de CSRF e sequestro de WebSockets.
- **Consistência de Dados (Likes):** Resolvida uma condição de corrida no sistema de likes, garantindo a integridade dos contadores de likes através de transações de base de dados.
- **Otimização de Performance (Paginação de Comentários):** Implementada paginação para os comentários, tanto no backend quanto no frontend, para melhorar a performance e a experiência do utilizador em posts com muitos comentários.

3. Detalhes das Correções por Categoria

3.1. Regressão de API: Conflito de `safeJSONParse`

Problema Original: O ficheiro `fixes.js` (carregado após `utils.js`) sobrescrevia a função `safeJSONParse` com uma assinatura incompatível. Enquanto `utils.js` esperava uma chave do `localStorage`, a versão em `fixes.js` esperava uma string JSON já lida. Isso resultava em `JSON.parse("users")` (em vez de `JSON.parse(localStorage.getItem("users"))`), que falhava e retornava um array vazio, levando à perda de dados em runtime.

Alterações Implementadas:

- **legacy_static_version/fixes.js** : A função `safeJSONParse` foi refatorada para detetar se o input é uma chave do `localStorage` ou uma string JSON. A lógica original que esperava uma string JSON foi renomeada para `safeJSONParseString`, e a função `safeJSONParse` principal agora atua como um wrapper que delega para `safeGetFromStorage` (se for uma chave) ou `safeJSONParseString` (se for uma string JSON).

Impacto: A aplicação agora consegue ler e persistir dados do `localStorage` corretamente, restaurando a funcionalidade essencial de carregamento de utilizadores, posts e outras informações.

3.2. Segurança de Autenticação: Hash de Password Inconsistente

Problema Original: O ficheiro `auth.js` utilizava a função `hashPassword` (de `utils.js`), que implementava um hash simples e vulnerável. Embora `fixes.js` contivesse uma versão `hashPasswordImproved`, esta não estava a ser utilizada no fluxo de autenticação real.

Alterações Implementadas:

- **legacy_static_version/auth.js** : O código foi modificado para utilizar `hashPasswordImproved` (se disponível) para novos registos e para a verificação de login. Foi adicionada lógica de compatibilidade para permitir que utilizadores registados com o hash antigo ainda consigam fazer login, facilitando uma transição suave.

Impacto: A segurança das passwords armazenadas no `localStorage` foi melhorada, tornando-as mais resistentes a ataques de força bruta, embora a recomendação final seja sempre usar hashing no backend com algoritmos como bcrypt ou Argon2.

3.3. Segurança DOM: Injeção de HTML Residual

Problema Original: Apesar de algumas correções anteriores, a função `displayFriendsList` em `app.js` ainda utilizava `innerHTML` com interpolação direta de strings para renderizar a lista de amigos. Embora `sanitizeInput` fosse usado, a prática de manipular o DOM diretamente com `innerHTML` é inerentemente mais arriscada e menos performática do que a criação programática de elementos DOM.

Alterações Implementadas:

- **legacy_static_version/app.js** : A função `displayFriendsList` foi refatorada para criar elementos DOM de forma programática (`document.createElement`) e anexá-los ao `friendsDiv` . Isso elimina a necessidade de `innerHTML` para a construção da lista de amigos.

Impacto: Redução do risco de vulnerabilidades de Cross-Site Scripting (XSS) e melhoria da robustez do código de renderização da UI.

3.4. Vulnerabilidade IDOR: Notificações

Problema Original: A rota `markAsRead` em `server/features.ts` permitia que um utilizador marcasse qualquer notificação como lida, bastando fornecer o `notificationId` , sem verificar se a notificação pertencia ao utilizador autenticado (`ctx.user.id`).

Alterações Implementadas:

- **server/queries.ts** : A função `markNotificationAsRead` foi atualizada para aceitar também o `userId` e incluir uma cláusula `WHERE` que verifica se a notificação pertence ao utilizador que a está a marcar como lida.
- **server/features.ts** : A rota `markAsRead` foi modificada para passar o `ctx.user.id` para a função `markNotificationAsRead` .

Impacto: A vulnerabilidade de IDOR foi mitigada, garantindo que os utilizadores só podem interagir com as suas próprias notificações.

3.5. Configuração Insegura de CORS e Cookies

Problema Original:

- **CORS no Socket.IO**: O Socket.IO estava configurado com `origin: "*" ,` permitindo ligações de qualquer origem, o que é inseguro em produção.
- **Cookies SameSite: "none"** : Os cookies de sessão tinham a política `SameSite: "none"` , o que os tornava vulneráveis a ataques de CSRF e facilitava o sequestro de

WebSockets em conjunto com a configuração de CORS.

Alterações Implementadas:

- **server/socket.ts** : A configuração de CORS para o Socket.IO foi atualizada para restringir as origens permitidas em ambiente de produção, utilizando a variável de ambiente `ALLOWED_ORIGINS`.
- **server/_core/cookies.ts** : A política `sameSite` para os cookies de sessão foi alterada de `"none"` para `"lax"`. Esta política oferece um bom equilíbrio entre segurança e funcionalidade, permitindo que os cookies sejam enviados em navegações de nível superior, mas não em requisições cross-site de terceiros.

Impacto: A segurança da aplicação foi significativamente reforçada contra ataques de CSRF e CSWH, protegendo as sessões dos utilizadores.

3.6. Consistência de Dados: Condição de Corrida nos Likes

Problema Original: O contador de likes (`posts.likes`) era atualizado com `sql ${posts.likes} + 1``, o que é suscetível a condições de corrida. Se múltiplos utilizadores dessem like/unlike ao mesmo tempo, o contador poderia ficar dessincronizado com o número real de entradas na tabela `postLikes`.

Alterações Implementadas:

- **server/queries.ts** : A função `toggleLike` foi refatorada para utilizar transações de base de dados. Dentro da transação, o número de likes é recalculado diretamente da tabela `postLikes` após cada operação de like/unlike, garantindo que o contador em `posts` reflita sempre o estado real.

Impacto: A integridade dos dados do contador de likes foi assegurada, eliminando a condição de corrida e fornecendo um valor preciso de likes para cada post.

3.7. Performance e UX: Paginação de Comentários

Problema Original: A função `getPostComments` no backend retornava todos os comentários de um post de uma só vez, o que poderia causar problemas de performance e consumo de memória para posts com um grande volume de comentários.

Alterações Implementadas:

- **server/queries.ts** : A função `getPostComments` foi atualizada para aceitar parâmetros `limit` e `offset`, permitindo a recuperação paginada de comentários.

- **server/features.ts** : A rota tRPC `posts.comments` foi modificada para aceitar os parâmetros de paginação e passá-los para `getPostComments` .
- **client/src/pages/Feed.tsx** : O componente `CommentsList` foi atualizado para implementar a lógica de paginação no frontend, carregando comentários em blocos e oferecendo um botão “Ver mais comentários” para o utilizador.

Impacto: Melhoria significativa na performance e na experiência do utilizador ao visualizar posts com muitos comentários, reduzindo o tempo de carregamento inicial e o consumo de recursos.

4. Conclusão

As intervenções realizadas abordaram uma série de problemas críticos e de alta severidade, transformando o projeto MINE-REDE-SOCIAL- numa aplicação mais segura, estável e performática. As correções foram validadas e o código-fonte atualizado no repositório GitHub. Recomenda-se a revisão contínua do código e a implementação de testes automatizados para garantir a manutenção da qualidade e segurança a longo prazo.